

Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain), **Advanced** (what actually keeps the Reasoner from quietly answering beyond what was found). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

The Reasoner replaces the `answer_placeholder` node Step 3 left as a stub. It's structurally simple — a chain, prompt in, model call, answer out, same shape as Project 2's Step 1 — but what goes into the prompt matters a lot more here: `state["found_chunks"]` (the Retriever's actual output), not the model's own memory of the topic. This node only ever runs on the `"answer"` branch of Step 3's routing, right after a `grounded=True` result — by the time it runs, the Retriever has already vouched for what it's about to see.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The function to build:

```
def reason_node(state: GraphState) -> dict:
```

Join `state["found_chunks"]`'s `page_content` fields into one block of context text, and build a prompt with a system message that names the rule explicitly: "answer the question using ONLY the context provided below — if the context doesn't contain the answer, say so plainly, don't fill the gap from what you already know." Pass `{context, task}` into the prompt, invoke the model, and return `{"messages": [AIMessage(content=answer)]}` (or a dedicated `draft_answer` state field, if you'd rather keep it separate from the raw message history — either is defensible, pick one and be consistent).

Test it by hand against a couple of real cases, as the README asks — not "did it run without error," but "does the actual wording of the answer trace back to the actual wording of the chunks," checked by eye.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

"Answer only from what was found" is an instruction the model can quietly ignore in one specific, easy-to-miss way: when the found chunks are *close* to the answer but don't quite contain it, a model will often blend the real chunk content with what it already knows from training, producing an answer that reads as fully grounded but is actually part-retrieved, part-remembered. This is worse than the model answering entirely from memory, because a part-fabricated answer is much harder to catch by eye — most of it really is supported. The instruction needs to be specific enough to block this blending, not just "use the context": something like "if any part of your answer isn't directly supported by the context below, leave that part out rather than filling it in."

The second gap, specific to this step's own boundary with Step 3: the Reasoner shouldn't be the one deciding whether the context is good enough — that's the Retriever's job, already done, one node earlier. A Reasoner that re-judges relevance (silently declining to answer even when `state["grounded"]` was already `True`) blurs a boundary the README is explicit about keeping separate ("did we find good context" vs. "did we use it correctly"). Trust `grounded=True` as a precondition of this node running at all, and only worry about *how the found content gets used*, not whether it should have been trusted in the first place.

The extra pieces: - System prompt language that explicitly forbids filling gaps from memory, not just "use the context" — the failure mode is partial blending, not wholesale ignoring. - A test case built specifically to blend-test this: a question where the found chunks cover *part* of the answer but not all of it, checking the draft explicitly says what it doesn't know instead of quietly completing the picture.

Write a test question you expect to trigger partial blending before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate wire the Reasoner in as a real node that uses the found chunks. Advanced is about the one failure mode "grounded" answers actually have in practice — not ignoring the context outright, but quietly blending it with memory when the context almost, but doesn't quite, cover the question.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
reason_node(state):
    context = join all found_chunks' text together

    prompt: "Answer using ONLY the context below. If the context doesn't
            contain the answer, say you don't have enough information."
    ask the model with {context, task}
    return the draft answer

wire: retrieve -> (grounded?) -> reason_node -> end
      \-> not grounded -> cant_ground_node -> end
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/reasoner_agent.py:
    REASONER_SYSTEM_PROMPT = """Answer the user's question using ONLY the
    context provided below. If the context does not contain the answer,
    say plainly that you don't have enough information – do not use
    anything you already know beyond what's in the context."""

    def reason_node(state):
        context = "\n---\n".join(c.page_content for c in state["found_chunks"])
        prompt = ChatPromptTemplate.from_messages([
            ("system", REASONER_SYSTEM_PROMPT),
            ("human", "Context:\n{context}\n\nQuestion: {task}"),
        ])
        model = ChatOpenAI(model="gpt-4o-mini")
        chain = prompt | model | StrOutputParser()
        answer = chain.invoke({"context": context, "task": state["task"]})
        return {"messages": [AIMessage(content=answer)]}

graph.py:
    builder.add_node("reason", reason_node)
    # replace "answer_placeholder" with "reason" everywhere it was wired in Step
```

Write the full typed version yourself, and test it by hand against 2 real questions — one clearly covered by your knowledge base, one only partially covered — before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

REASONER_SYSTEM_PROMPT adds: "If any part of your answer is not directly supported by the context below, leave that part out rather than filling it in from what you already know. It's better to give a partial answer than a complete-sounding one that isn't fully grounded."

Here's almost the whole thing — fill in the missing piece yourself:

```
REASONER_SYSTEM_PROMPT = """Answer the user's question using ONLY the context
provided below. If the context does not contain the answer, say plainly that
you don't have enough information. If any part of your answer is not directly
supported by the context, leave that part out rather than filling it in from
what you already know."""
```

```
def reason_node(state):
    context = "\n---\n".join(c.page_content for c in state["found_chunks"])
    # your turn: build and invoke the chain exactly as in Intermediate,
    # using REASONER_SYSTEM_PROMPT above
    ...

# your turn: write a test question where found_chunks cover PART of the
# answer but not all of it – check the draft admits what it doesn't cover
def test_reasoner_does_not_blend_memory_with_partial_context():
    ...
```

Fill in `reason_node`'s body and the partial-coverage test, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the chain's shape (context + task in, grounded answer out) is identical across all 3 levels. Advanced sharpens the one instruction that actually matters once a real knowledge base is used — not "use the context," which every level already says, but "don't blend the context with what you already know when it's only a partial match," which is the specific, easy-to-miss way grounding quietly leaks.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)